

15.1.1. Car Dealership Sales System

Algorithm: Car Dealership Sales System

- 1. Start**
- 2. Define Base Class Car**
 - Create a constructor `__init__()` with attributes:
 - brand
 - price
 - model
 - color
 - Define method `display_details()`:
 - Print brand
 - Print price
 - Print model
 - Print color
- 3. Define Derived Class Car1**
 - Inherit from Car
 - Override `display_details()` (if required format is different)
 - Otherwise, use parent method
- 4. Define Derived Class Car2**
 - Inherit from Car
 - Override `display_details()` (if required)
 - Otherwise, use parent method
- 5. Take Input for Car1**
 - Read input line
 - Split into:
 - brand1, price1, model1, color1
 - Convert price1 to float
- 6. Take Input for Car2**

- Read input line
- Split into:
 - brand2, price2, model2, color2
- Convert price2 to float

7. Create Objects

- Create object car1 using Car1(brand1, price1, model1, color1)
- Create object car2 using Car2(brand2, price2, model2, color2)

8. Display Details

- Call car1.display_details()
- Call car2.display_details()

9. End

Programme :

15.1.1. Car Dealership Sales System

Write a Python program to model a car dealership sales system using object-oriented programming. Create a base class `Car` and two derived classes `Car1` and `Car2` to represent different car types.

Class Structure

- Base Class `Car`
 - Attributes: `brand`, `price`, `model`, `color`.
 - Method: `display_details()` to print all car attributes.
- Derived Classes `Car1` and `Car2`
 - Inherit from `Car` class.
 - Override `display_details()` method to display car information in the specified format.

Input Format:

- The first line contains `Car1`'s `brand` (string), `price` (float), `model` (string), and `color` (string) separated by spaces.
- The second line contains `Car2`'s `brand` (string), `price` (float), `model` (string), and `color` (string) separated by spaces.

Output Format:

- First four lines: `Car1` details (`brand`, `price`, `model`, `color` - each on separate line)
- Next four lines: `Car2` details (`brand`, `price`, `model`, `color` - each on separate line)

Sample Test Cases

```

12 # Display details
13 car1.display_details()
14 car2.display_details()"""
15 class Car:
16     def __init__(self, brand, price, model, color):
17         self.brand = brand
18         self.price = price
19         self.model = model
20         self.color = color
21
22     def display_details(self):
23         print(self.brand)
24         print(self.price)
25         print(self.model)
26         print(self.color)
27
28     class Car1(Car):
29         pass
30     class Car2(Car):
31         pass
  
```

Test Case Results:

- 3 out of 3 shown test case(s) passed
- 3 out of 3 hidden test case(s) passed

Test case 1

Expected output	Actual output
Toyota 25000.50 Camry White	Toyota 25000.50 Camry White
Honda 22000.00 Accord Black	Honda 22000.00 Accord Black
Toyota	Toyota
25000.5	25000.5
Camry	Camry
White	White

Flowchart :

